

2026년도 전국기능경기대회 채점기준

1. 채점상의 유의사항	직 종 명	클라우드컴퓨팅
※ 다음 사항을 유의하여 채점하십시오. 1) AWS의 리전은 ap-northeast-2를 사용합니다. 2) 웹페이지 접근은 크롬이나 파이어폭스를 이용합니다. 3) 웹페이지에서 언어에 따라 문구가 다르게 보일 수 있습니다. 4) shell에서의 명령어의 출력은 버전에 따라 조금 다를 수 있습니다. 5) 문제지와 채점지에 있는 <math>\diamond</math> 는 변수입니다. 해당 부분을 변경해 입력합니다. 6) 채점은 문항 순서대로 진행해야 합니다. 7) 삭제된 채점자료는 되돌릴 수 없으므로 유의하여 진행하며, 이의신청까지 완료 이후 선수가 생성한 클라우드 리소스를 삭제합니다. 8) 부분 점수가 있는 문항은 채점 항목에 부분 점수가 적혀져 있습니다. 9) 부분 점수가 따로 없는 문항은 모두 맞아야 점수로 인정됩니다. 10) 리소스의 정보를 읽어오는 채점항목은 기본적으로 스크립트 결과를 통해 채점을 진행하며, 만약 선수가 이의가 있다면 명령어를 직접 입력하여 확인해볼 수 있습니다. 11) (예상 출력)은 바로 이전 (명령어 입력)의 예상 출력을 의미합니다. 12) 채점 시에는 별도로 제공한 채점 스크립트(mark.sh)를 실행하여 채점할 수 있습니다. 다만, 선수가 직접 입력을 원할 경우 채점기준표에 명시된 명령어 그대로 입력하여 채점할 수 있습니다. 13) 제공된 채점 스크립트는 /home/cloudshell-user 경로에 위치하도록 합니다. 14) 모든 채점은 서울 리전의 CloudShell VPC Environment unicorn-mark에서 진행합니다. 15) 채점지에서 페이지 간 줄바꿈되어 일부 잘린 항목이 있을 수 있음에 유의합니다. 16) 별도 명시가 없는 경우, 빨간색으로 표시된 부분은 정확히 일치하는지 확인하고, 파란색으로 표시된 부분은 무시할 수 있습니다. 17) 스크립트 출력 값이 예상 출력 값에서 줄바꿈되어 나타나는 등은 무시합니다.		

2. 채점기준표

1) 주요항목별 배점			직 종 명		클라우드컴퓨팅			
과제 번호	일련 번호	주요항목	배점	채점방법		채점시기		비고
				독립	합의	경기 진행중	경기 종료후	
제1과제	1	Networking	3.0	○			○	
	2	KMS	1.0	○			○	
	3	S3	1.0	○			○	
	4	Database	1.5	○			○	
	5	ECR	1.0	○			○	
	6	EKS	4.5	○			○	
	7	Lambda	1.0	○			○	
	8	Service Endpoint	7.0	○			○	
	9	Security	2.0	○			○	
	10	Application	1.5	○			○	
	11	Observability	2.0	○			○	
	12	Runtime Test	3.0	○			○	
	13	Grafana	1.5	○			○	
합 계			30					

2) 채점방법 및 기준

과제 번호	일련 번호	주요항목	일련 번호	세부항목 (채점방법)	배점
1과제	1	Networking	1	VPC & Subnet CIDR Configuration	1.0
			2	Routing Configuration	1.0
			3	VPC Endpoint & Flow Log	1.0
	2	KMS	1	Keys & Rotation Configuration	1.0
	3	S3	1	S3 bucket Configuration	1.0
	4	Database	1	DynamoDB Table Configuration	1.5
	5	ECR	1	ECR Repository Configuration	1.0
	6	EKS	1	EKS Cluster Configuration	1.5
			2	EKS NodeGroup Configuration	1.5
			3	Workload Check	1.5
	7	Lambda	1	Lambda Function Configuration	1.0
	8	Service Endpoint	1	ALB Routing Configuration	1.0
			2	CloudFront CDN Configuration	1.5
			3	Book app POST Request Test	1.5
			4	Book app GET Request Test	1.5
			5	ALB Direct Request deny Test	0.5
			6	WAF Managed Rule Test	1.0
	9	Security	1	Audit Role Configuration	0.5
			2	Audit Role Assume and Permission Test	1.5
	10	Application	1	Application Health and env, etc	1.5
	11	Observability	1	Log type & Health log exclusion	1.5
			2	Prometheus Metrics	0.5
	12	Runtime Test	1	Log Pipeline Test	1.5
			2	WAF Rate limit block Test	1.5
	13	Grafana	1	Grafana Dashboard Panel Configuration	1.5

3) 채점내용

순번	사전준비	
0	0) 채점 전, 채점 유의사항을 정독합니다. 유의사항 16번을 확인 후 진행합니다. 1) unicorn-mark CloudShell VPC Environment에 접근합니다. 2) <code>rm -rf ~/.aws</code>를 진행합니다. 3) <code>aws configure</code>를 입력하고 <code>default.region</code>을 <code>ap-northeast-2</code>으로 설정합니다. 4) 채점 명령어에 사용될 환경 변수를 설정합니다. \$ export number= <선수등번호> 5) 이후, 아래 명령어를 통해 Kubectl context 등을 설정합니다 \$ source kubectl-connect unicorn-eks-cluster	
순번	채점항목	
1	1-1-A (명령어 입력)	<pre>aws ec2 describe-vpcs --filters Name= tag:Name,Values= unicorn-vpc -- query "Vpcs[].CidrBlock" -- output json jq -r "[]" aws ec2 describe-subnets --filters Name= tag:Name,Values= unicorn-subnet-pub-a,unicorn-subnet-pu b-b,unicorn-subnet-pub-c,unicorn-subnet-priv-a,unicorn-subnet- priv-b,unicorn-subnet-priv-c -- query "Subnets[].CidrBlock" -- output text</pre>
	1-1-A (예상 출력) <u>순서 무관</u>	<pre>10.97.0.0/16 10.97.10.0/24 10.97.0.0/24 10.97.2.0/24 10.97.12.0/24 10.97.11.0/24 10.97.1.0/24</pre>
	1-2-A (명령어 입력)	<pre>aws ec2 describe-route-tables --filters "Name= tag:Name,Values= unicorn-rt-*" --query "RouteTables[][Tags[?Key= 'Name']][0].Value, Routes[?DestinationCidrBlock= '0.0.0.0/0']][0].GatewayId, Routes[?DestinationCidrBlock= '0.0.0.0/0']][0].NatGatewayId, length(Associations[?SubnetId!= null])" --output text</pre>
	1-2-A (예상 출력) <u>부분 일치</u> <u>순서 무관</u>	<pre>unicorn-rt-priv-b None nat-0d7aac9f8d35d2289 1 unicorn-rt-pub igw-086901b749303496a None 3 unicorn-rt-priv-a None nat-05cab70f28b9ad467 1 unicorn-rt-priv-c None nat-0bc21e61017b3eae5 1</pre>
	1-3-A (명령어 입력)	<pre>VPC_ID=\$(aws ec2 describe-vpcs --filters Name= tag:Name,Values= unicorn-vpc --query "Vpcs[0].VpcId" --output text) aws ec2 describe-vpc-endpoints --filters Name= vpc-id,Values= \$VPC_ID --query "VpcEndpoints[].ServiceName" --output json jq -r "[]"</pre>

	aws ec2 describe-flow-logs --filter Name= resource-id,Values= \$VPC_ID --query "length(FlowLogs)" --output text
1-3-A (예상 출력)	com.amazonaws.ap-northeast-2.s3 com.amazonaws.ap-northeast-2.ecr.api com.amazonaws.ap-northeast-2.ecr.dkr 1 <u>* 출력값에 s3, ecr.api, ecr.dkr가 포함되어 있고, 1 이상이면 득점.</u>
2-1-A (명령어 입력)	for a in app data platform; do aws kms get-key-rotation-status --key-id \$(aws kms describe-key --key-id alias/unicorn-kms-\$a --query "KeyMetadata.KeyId" --output text) --query "[KeyRotationEnabled, RotationPeriodInDays]" --output text; done
2-1-A (예상 출력) <u>정확히 일치</u>	True 90 True 90 True 90
3-1-A (명령어 입력)	ACCOUNT_ID=\$(aws sts get-caller-identity --query Account --output text) BUCKET= unicorn-web-\$ACCOUNT_ID aws s3api list-buckets --query "Buckets[?contains(Name, 'unicorn-web-')].Name" jq -r '[]' aws s3api get-public-access-block --bucket \$BUCKET --query "PublicAccessBlockConfiguration" --output json jq -r 'to_entries map(value) @tsv' aws s3api get-bucket-versioning --bucket \$BUCKET --query "Status" --output text aws s3api get-bucket-encryption --bucket \$BUCKET --query "ServerSideEncryptionConfiguration.Rules[0].ApplyServerSideEncryptionB yDefault.[SSEAlgorithm, KMSMasterKeyID]" --output text
3-1-A (예상 출력) <u>부분 일치</u>	unicorn-web-837933860870 true true true true Enabled aws:kms arn:aws:kms:ap-northeast-2:837933860870:key/2ec8a562-b337-4f43
4-1-A (명령어 입력)	aws dynamodb describe-table --table-name unicorn-concert-db --query "Table.{Billing:BillingModeSummary.BillingMode, PK:KeySchema[?KeyType= 'HASH'].AttributeName[0],

	GSIName:GlobalSecondaryIndexes[0].IndexName, GSI_PK:GlobalSecondaryIndexes[0].KeySchema[?KeyType= 'HASH'].AttributeName[0], GSI_SK:GlobalSecondaryIndexes[0].KeySchema[?KeyType= 'RANGE'].AttributeName[0], GSIProj:GlobalSecondaryIndexes[0].Projection.ProjectionType, SSEType:SSEDescription.SSEType, SSEKms:SSEDescription.KMSMasterKeyArn, Delete:DeletionProtectionEnabled}" --output json aws dynamodb describe-continuous-backups --table-name unicorn-concert-db --query "ContinuousBackupsDescription.PointInTimeRecoveryDescription.PointInTimeRecoveryStatus" --output text
4-1-A (예상 출력) <u>부분 일치</u>	{ "Billing": "PAY_PER_REQUEST", "PK": "booking_id", "GSIName": "client-id-created-at-index", "GSI_PK": "client_id", "GSI_SK": "created_at", "GSIProj": "ALL", "SSEType": "KMS", "SSEKms": "arn:aws:kms:ap-northeast-2: 계정 ID:key/6fe58a37-030 ", "Delete": true } ENABLED
5-1-A (명령어 입력)	aws ecr describe-repositories --repository-names unicorn-concert-app --query "repositories[0].{Scan:imageScanningConfiguration.scanOnPush, Mutability:imageTagMutability, Enc:encryptionConfiguration.encryptionType}" --output json jq -r "[]" aws ecr describe-images --repository-name unicorn-concert-app --query "sort(imageDetails[].imageTags[])" --output json jq -r '@tsv' aws ecr describe-image-scan-findings --repository-name unicorn-concert-app --image-id imageTag=v1.0.0 --query "imageScanFindings.findingSeverityCounts" --output json jq .
5-1-A (예상 출력) <u>부분 일치</u>	true IMMUTABLE_WITH_EXCLUSION KMS latest v1.0.0 * 버전이 더 존재하는 것은 무시합니다.

	<u>* 버전 태그 아래 취약점 개수가 출력되지 않아야 합니다.</u>
6-1-A (명령어 입력)	<pre>aws eks describe-cluster --name unicorn-eks-cluster --query "cluster.version" --output text aws eks describe-cluster --name unicorn-eks-cluster --query "cluster.resourcesVpcConfig[endpointPublicAccess, endpointPrivateAccess]" --output json jq -r '@tsv' aws eks describe-cluster --name unicorn-eks-cluster --query "cluster.logging.clusterLogging[?enabled==\ `true\ `].types[]" --output json jq -r '@tsv' aws eks describe-cluster --name unicorn-eks-cluster --query "cluster.encryptionConfig[.provider.keyArn]" --output text aws eks describe-cluster --name unicorn-eks-cluster --query "cluster.accessConfig.authenticationMode" --output text</pre>
6-1-A (예상 출력) <u>부분 일치</u>	<pre>1.35 false true api audit authenticator controllerManager scheduler arn:aws:kms:ap-northeast-2:837933860870:key/b54d1980-5bc9-48 API</pre>
6-2-A (명령어 입력)	<pre>kubectrl get nodes -l unicorn=app -o jsonpath='{range .items[*]}{.metadata.labels.topology\.kubernetes\.io/zone}{\ `n`}{'\ `end}' kubectrl get nodes -l unicorn=addon --no-headers wc -l aws ec2 describe-instances --filters Name= tag:Name,Values= unicorn-k8snode-app-node Name= instance-state-name,Values= running --query "length(Reservations[].Instances[])" --output text aws ec2 describe-instances --filters Name= tag:Name,Values= unicorn-k8snode-addon-node Name= instance-state-name,Values= running --query "length(Reservations[].Instances[])" --output text aws ec2 describe-instances --filters Name= tag:Name,Values= unicorn-k8snode-app-node Name= instance-state-name,Values= running --query "Reservations[].Instances[.PublicIpAddress]" --output json</pre>
6-2-A (예상 출력) <u>부분 일치</u>	<pre>ap-northeast-2a <- 해당 부분의 2개 이상 AZ 출력되면 정답 ap-northeast-2b 1 2 1</pre>

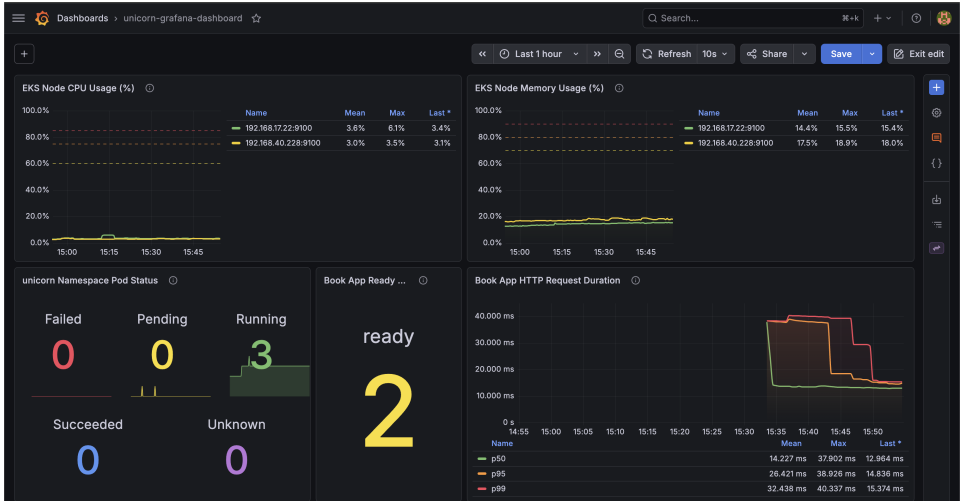
	□ * 빨간색 부분의 경우 1 이상, 파란색 부분의 경우 2 이상이면 정답
6-3-A (명령어 입력)	<pre>kubectl get deploy unicorn-book-app-deploy -n unicorn -o custom-columns='NAME:.metadata.name,READY:.status.readyReplicas,AV AILABLE:.status.availableReplicas' --no-headers kubectl get svc unicorn-book-app-svc -n unicorn -o custom-columns='NAME:.metadata.name,TYPE:.spec.type' --no-headers kubectl get deploy unicorn-book-app-deploy -n unicorn -o jsonpath='{liveness={.spec.template.spec.containers[0].livenessProbe.httpG et.path} readiness={.spec.template.spec.containers[0].readinessProbe.httpGet.path }\ n"}graceful={.spec.template.spec.terminationGracePeriodSeconds} preStop={.spec.template.spec.containers[0].lifecycle.preStop}\ n"}' kubectl get pods -n unicorn -l app -o jsonpath='{range .items[*]}.{.spec.nodeSelector.unicorn}\ n"}{end}' sort -u aws eks list-pod-identity-associations --cluster-name unicorn-eks-cluster --namespace unicorn --query "associations[].serviceAccount" --output text</pre>
6-3-A (예상 출력) <u>부분 일치</u>	<pre>unicorn-book-app-deploy 2 2 <- 빨간색 부분 두 값이 같다면 정답 unicorn-book-app-svc liveness=/health readiness=/health graceful=45 preStop={"exec":{"command":["/bin/sh","-c","sleep 15"]}} app unicorn-book-app-sa <- 출력되는 값이 있다면 정답</pre>
7-1-A (명령어 입력)	<pre>aws lambda get-function-configuration --function-name unicorn-get-booking-func --query "[FunctionName, KMSKeyArn, LoggingConfig.LogGroup]" --output json jq -r ".[]"</pre>
7-1-A (예상 출력) <u>정확히 일치</u>	<pre>unicorn-get-booking-func arn:aws:kms:ap-northeast-2:837933860870:key/b54d1980-5bc9-482f /unicorn/lambda/get-booking</pre>
8-1-A (명령어 입력)	<pre>ALB_ARN=\$(aws elbv2 describe-load-balancers --names unicorn-alb --query "LoadBalancers[0].LoadBalancerArn" --output text) aws elbv2 describe-load-balancers --load-balancer-arns \$ALB_ARN --query "LoadBalancers[0].[Scheme, Type, State.Code]" --output text aws elbv2 describe-listeners --load-balancer-arn \$ALB_ARN --query "Listeners[0].[Protocol, Port]" --output text aws elbv2 describe-target-groups --names unicorn-tg --query "TargetGroups[0].TargetGroupName" --output text</pre>

8-1-A (예상 출력) <u>정확히 일치</u>	internal application active HTTP 80 unicorn-tg
8-2-A (명령어 입력)	aws cloudfront get-distribution-config --id \$(aws cloudfront list-distributions --query "DistributionList.Items[?Comment= 'unicorn-svc-cf'].Id [0]" --output text) --query "DistributionConfig.Origins.Items[].Id, OriginAccessControlId, VpcOriginConfig.VpcOriginId]" --output text aws s3api get-bucket-policy --bucket unicorn-web-\$(aws sts get-caller-identity --query Account --output text) --query "Policy" --output text jq -r '.Statement[] .Principal.Service, .Condition.StringEquals.AWS:SourceArn'
8-2-A (예상 출력) <u>강조 부분</u> <u>일치</u>	s3-origin E5QU162JYR0HC None < - <u>Origin 2개 존재할 경우 정답</u> app-origin vo_CYulQTnExqrDBT6a97EQ6T cloudfront.amazonaws.com arn:aws:cloudfront::837933860870:distribution/E1FBZ2QNYFHPN1
8-3-A (명령어 입력)	CF=\$(aws cloudfront list-distributions --query "DistributionList.Items[?Comment= 'unicorn-svc-cf'].DomainName [0]" --output text) RESP=\$(curl -s -X POST "https://\$CF/v1/book" -H 'Content-Type: application/json' -d '{"client_id": "C-MARK", "username": "Judge", "email": "judge@skills.kr", "concert_name": "UnicornMark2026"}') echo "\$RESP" BID=\$(echo "\$RESP" jq -r '.booking_id') aws dynamodb get-item --table-name unicorn-concert-db --key "{ \"booking_id\": \"\$BID\" }" --query "Item.{booking_id:booking_id.S, client_id:client_id.S, concert_name:concert_name.S, created_at:created_at.S}" --output json
8-3-A (예상 출력) <u>정확히 일치</u>	{"booking_id": " 1DBMHYLO "} { "booking_id": " 1DBMHYLO ", "client_id": " <u>C-MARK</u> ", "concert_name": " <u>UnicornMark2026</u> ", "created_at": "2026-05-31T20:00:59Z" } * <u>빨간색 부분이 서로 일치하고, 밑줄 친 부분이 채점기준표와 일치하면 득점</u>

8-4-A (명령어 입력)	curl -s "https://\$CF/v1/book?booking_id=\$BID" jq .
8-4-A (예상 출력) <u>정확히 일치</u>	<pre>{ "username": "Judge", "created_at": "2026-05-31T20:00:59Z", "email": "judge@skills.kr", "booking_id": "1DBMHYLO", "client_id": "C-MARK", "concert_name": "UnicornMark2026" }</pre> <u>* 강조한 부분의 8-3-A의 출력값과 같으면 득점. 8-3 틀렸을 경우 오답.</u>
8-5-A (명령어 입력)	curl -s -o /dev/null -w "%{http_code} n" --max-time 10 -X POST "http://\$(aws elbv2 describe-load-balancers --names unicorn-alb --query "LoadBalancers[0].DNSName" --output text)/v1/book" -H 'Content-Type: application/json' -d '{"client_id":"DIRECT"}' echo "000"
8-5-A (예상 출력) <u>정확히 일치</u>	000 000 * 또는 403이 출력되었을 경우 득점
8-6-A (명령어 입력)	curl -s -o /dev/null -w "%{http_code}" "https://\$CF/?probe=<script>alert(1)</script>"
8-6-A (예상 출력)	403
9-1-A (명령어 입력)	<pre>aws iam get-role --role-name unicorn-audit-role --output json jq -r '.Role [.MaxSessionDuration, .AssumeRolePolicyDocument.Statement[0].Principal.AWS, .AssumeRolePolicyDocument.Statement[0].Condition.StringEquals["sts:Ext ernalId"]]' map(tostring) join(" ") for p in \$(aws iam list-role-policies --role-name unicorn-audit-role --query "PolicyNames[]" --output text); do aws iam get-role-policy --role-name unicorn-audit-role --policy-name \$p --query "PolicyDocument.Statement[].Action[]" --output text; done</pre>
9-1-A (예상 출력) <u>부분 일치</u>	3600 arn:aws:iam::111122223333:root unicorn-audit-2026<선수등번호> dynamodb:GetItem dynamodb:Query ec2:DescribeVpcs eks:Describe <u>* 권한 정책의 경우 해당 부분 포함하며, *이 없으면 득점 인정.</u>
9-2-A (명령어 입력)	ACCOUNT_ID=\$(aws sts get-caller-identity --query Account --output text)

	<pre> ROLE_ARN=arn:aws:iam::\$(echo \$ACCOUNT_ID):role/unicorn-audit-role aws sts assume-role --role-arn \$ROLE_ARN --role-session-name mk 2>&1 grep -oE AccessDenied head -1 read -r AK SK TK < <(aws sts assume-role --role-arn \$ROLE_ARN --role-session-name mk --external-id unicorn-audit-2026\$number --query "Credentials.[AccessKeyId,SecretAccessKey,SessionToken]" --output text) export AWS_ACCESS_KEY_ID=\$AK AWS_SECRET_ACCESS_KEY=\$SK AWS_SESSION_TOKEN=\$TK aws sts get-caller-identity --query Arn --output text aws ec2 describe-vpcs --filters Name=tag:Name,Values=unicorn-vpc --query "Vpcs[0].VpcId" --output text aws ec2 describe-instances 2>&1 grep -oE "AccessDenied UnauthorizedOperation" head -1 unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN </pre>
9-2-A (예상 출력) <u>부분 일치</u>	<pre> AccessDenied arn:aws:sts::계정ID:assumed-role/unicorn-audit-role/mk vpc-08cc0ca87bdb7d71f UnauthorizedOperation </pre>
10-1-A (명령어 입력)	<pre> curl -s -o /dev/null -w "%{http_code}\n" "https://\$(aws cloudfront list-distributions --query "DistributionList.Items[?Comment='unicorn-svc-cf'].DomainName [0]" --output text)/health" kubectl exec -n unicorn \$(kubectl get pods -n unicorn -l app=book -o jsonpath='{.items[0].metadata.name}') -c book -- printenv AWS_REGION TABLE_NAME </pre>
10-1-A (예상 출력) <u>정확히 일치</u>	<pre> 200 ap-northeast-2 unicorn-concert-db </pre>
11-1-A (명령어 입력)	<pre> aws logs get-log-events --log-group-name /unicorn/eks/book-app --log-stream-name "\$(aws logs describe-log-streams --log-group-name /unicorn/eks/book-app --order-by LastEventTime --descending --limit 1 --query "logStreams[0].logStreamName" --output text)" --limit 1 --start-from-head --query "events[-1].message" --output text jq -r 'keys_unsorted sort join(",")' aws logs filter-log-events --log-group-name /unicorn/eks/book-app </pre>

		--filter-pattern ""/health"" --query "events[].message" --output text grep -c .
11-1-A (예상 출력) <u>정확히 일치</u>		client_ip,method,path,status_code,timestamp 0
11-2-A (명령어 입력)		kubectl get pods -n monitoring -o custom-columns= 'NAME:.metadata.name,STATUS:.status.phase' --no-headers grep -iE "prometheus-grafana" kubectl get servicemonitor -A -o jsonpath='{range .items[*]}{.metadata.name}{ " n"}{end}' 2>/dev/null grep -iE "kube-controller-manager kube-scheduler kube-etcd" wc -l
11-2-A (예상 출력) <u>정확히 일치</u>		* 강조된 부분이 일치해야 합니다. (Suffix, 개수는 다를 수 있음) prometheus -unicorn-monitoring-kube-pr-prometheus-0 Running unicorn-monitoring- grafana -7974ccf57f-j585v Running 0
12-1-A (명령어 입력) <u>(대기 포함)</u>		* 요청을 보낸 후, 로그 수집까지 30초 대기한 후 조회합니다. date -u "+ %Y-%m-%dT%H:%M:%SZ" SINCE=\$(date +%s) curl -s -X POST "https://\$(aws cloudfront list-distributions --query "DistributionList.Items[?Comment= 'unicorn-svc-cf'].DomainName [0]" --output text)/v1/book" -H 'Content-Type: application/json' -d '{"client_id": "C-FRESH", "username": "Fresh", "email": "fresh@skills.kr", "concer t_name": "FreshMark"}' > /dev/null echo "waiting 30s for log pipeline" && sleep 30 aws logs filter-log-events --log-group-name /unicorn/eks/book-app --start-time \${SINCE}000 --filter-pattern '{ \$.method = "POST" && \$.path = "/v1/book" }' --query "events[-1].message" --output text
12-1-A (예상 출력) <u>부분 일치</u>		2026-05-31T23:54:40Z waiting 30s for log pipeline { "timestamp": " 2026-05-31T23:54:42Z ", "method": "POST", "path": "/v1/book" , "status_code": 200, "client_ip": " 10.97.12.184 " } * 출력값 형식이 위와 같고, 빨간색 부분의 시간 차가 1분 이내면 득점.
12-2-A (명령어 입력) (대기 포함)		* 60초 대기한 후 부하를 주입합니다. 이후 30초 대기합니다. 대기가 끝난 후 선수는 30초 간 원하는 만큼 curl 요청을 할 수 있습니다. CF=\$(aws cloudfront list-distributions --query "DistributionList.Items[?Comment= 'unicorn-svc-cf'].DomainName [0]" --output text)

	<pre> sleep 60 for i in \$(seq 1 100); do curl -s -o /dev/null "https://\$CF/health"; done sleep 30 curl -s -o /dev/null -w "%{http_code}\n" "https://\$CF/health" curl -s "https://\$CF/health" </pre>
12-2-A (예상 출력) 정확히 일치	403 Request blocked by Unicorn WAF
13-1-A (수동 채점)	unicorn-grafana-alb에 접근하여 skills<선수등번호> / HelloKrSkills!<등번호>@로 로그인합니다. unicorn-grafana-dashboard로 이동한 후, 아래와 같이 대시보드가 잘 구성되었는지 확인합니다. 만약 Panel type 등이 다르거나, No Data가 있는 경우에도 오답입니다. 대소문자는 채점 시 고려하지 않습니다. 패널 구성은 아래 이미지를 참조하나, 필요 시 아래 설명을 참고합니다. * 올바른 Panel 구성 1. EKS Node CPU Usage (%) - Time Series 2. EKS Node Memory Usage (%) - Time Series 3. unicorn Namespace Pod Status - Stat, graph 포함 4. Book App Ready Pods - Stat 5. Book App HTTP Request Duration - Time Series * <u>4번은 Book App Ready까지만 출력되어도 정답.</u>  선수가 색상, 대소문자 등을 다르게 구성한 것은 정답 처리하나, 텍스트를 다르게 작성했거나, 위 사진에 나온 Panel 중 하나라도 없는

	등의 차이가 있다면 오답 처리. * 표시되는 값의 경우 채점 시 무시함. 선수마다 다르게 출력됨.
--	--